



Клиентские технологии безопасности

Александр Менщиков,
к.т.н., доцент ИТМО

Содержание лекции

1. Основы JavaScript
2. Уязвимость XSS
3. Same-origin policy
4. Cross-Origin Resource Sharing (CORS)
5. Content Security Policy (CSP)
6. Уязвимость CSRF

Основы JavaScript

Что такое JavaScript?

<https://learn.javascript.ru/>

<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

1. Движок браузера парсит текст JavaScript
2. Скрипт компилируется в машинный язык
3. Машинный код запускается в песочнице браузера

Что может JS (еще раз)

Добавлять новый HTML-код на страницу, изменять существующее содержимое, модифицировать стили.

```
<body><h1>Empty</h1></body>
```

```
<script>
```

```
    document.getElementsByTagName("h1")[0].innerText = "Modified Header";  
    document.getElementsByTagName("h1")[0].style = "background-color:#f74d09";  
    const newDiv = document.createElement('div');  
    newDiv.innerHTML = "<h3>Subtitle</h3>";  
    document.body.appendChild(newDiv);
```

```
</script>
```

Modified Header

Subtitle

Что может JS (еще раз)

Реагировать на действия пользователя, щелчки мыши, перемещения указателя, нажатия клавиш.

```
<body>
  <div id="x"></div>
  <div id="y"></div>
</body>
<script>
  onmousemove = function (e) {
    document.getElementById("x").innerText = `X: ${e.clientX}`;
    document.getElementById("y").innerText = `Y: ${e.clientY}`;
  }
</script>
```

X: 55
Y: 44

Что может JS (еще раз)

Отправлять сетевые запросы на удалённые сервера, скачивать и загружать файлы.

```
<button onclick="setInterval(updateDateTime, 1000)">Start timer</button>  
<div id="time"></div>
```

```
<script>  
async function updateDateTime() {  
    document.getElementById("time").innerText = await  
        fetch("https://cs.itmo.ru/webtech/time").then(res => res.text())  
}  
</script>
```

Что может JS (еще раз)

Получать и устанавливать куки, задавать вопросы посетителю, показывать сообщения.

```
<meta charset="utf-8" />
<script>
  let cookie = prompt("Какую Cookie хотите установить?", "test");
  document.cookie = `${cookie}=supersecret;`;
  let result = confirm("Хотите увидеть свои Cookie?");
  if (result) {
    alert(`Cookie: ${document.cookie}`);
  }
  test=supersecret; test2=supersecret; null=supersecret
</script>
```


Уязвимость XSS

Cross Site Scripting (XSS)

Атака, заключающаяся во внедрении на страницу сайта или приложения вредоносного кода. Когда пользователь открывает такую страницу, внедренный скрипт выполняется.

- Исполнение произвольного кода (JavaScript)
- Проведение других атак (CSRF)
- Кража Cookie и/или конфиденциальной информации
- Модификация визуального представления страницы

Разновидность №1 – Reflected XSS

Отраженные XSS – XSS, в котором полезная нагрузка содержится в HTTP запросе пользователя.

Например, злоумышленник прячет вредоносный код в ссылке (URL), которую потом передает пользователям-жертвам. Когда пользователь переходит по такой ссылке, серверные скрипты исполняют ее код.

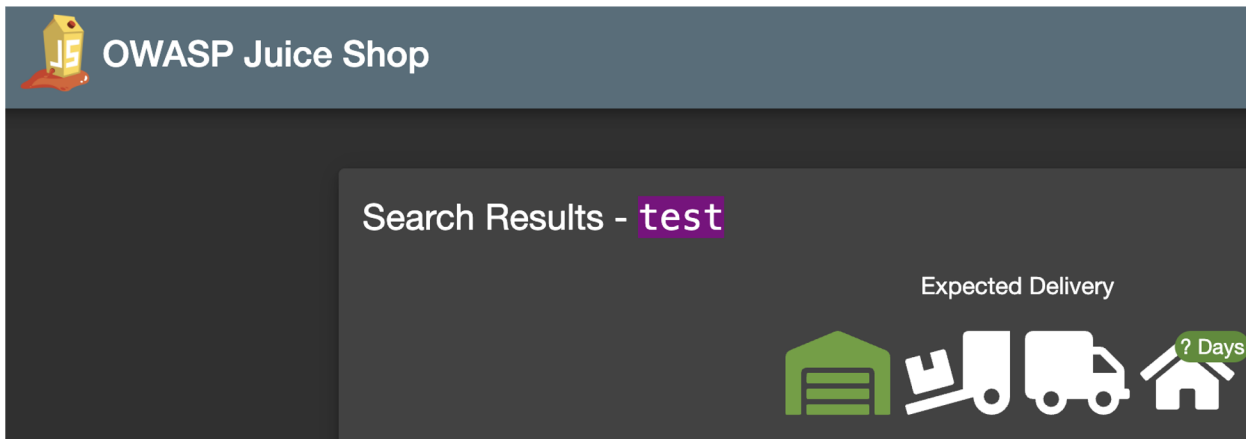
- > Что нужно для выполнения?
- > Ошибочно написанные JS на сайте, которые случайно исполнят код из ссылки

Reflected XSS пример

```
docker run --rm -p 127.0.0.1:3000:3000 -e NODE_ENV=unsafe bkimminich/juice-shop
```

1. Нашли место, где **передаваемые нами данные**, отображаются **без фильтрации** на странице

```
http://localhost:3000/#/track-result/new?id=<span style="background:purple">test</span>
```



Reflected XSS пример

2. Подбираем payload, который откроет alert()

<https://github.com/payloadbox/xss-payload-list>

<https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>

```
<img/src/onerror=alert(xss)>
```

3. Тестируем payload на себе, добиваемся правильного результата и отправляем жертве ссылку:

```
http://localhost:3000/#/track-  
result/new?id=%3Cimg%2Fsrc%2Fonerror%3Dalert(%22xss%22)%3E
```

Разновидность №2 – Stored XSS (persisted XSS)

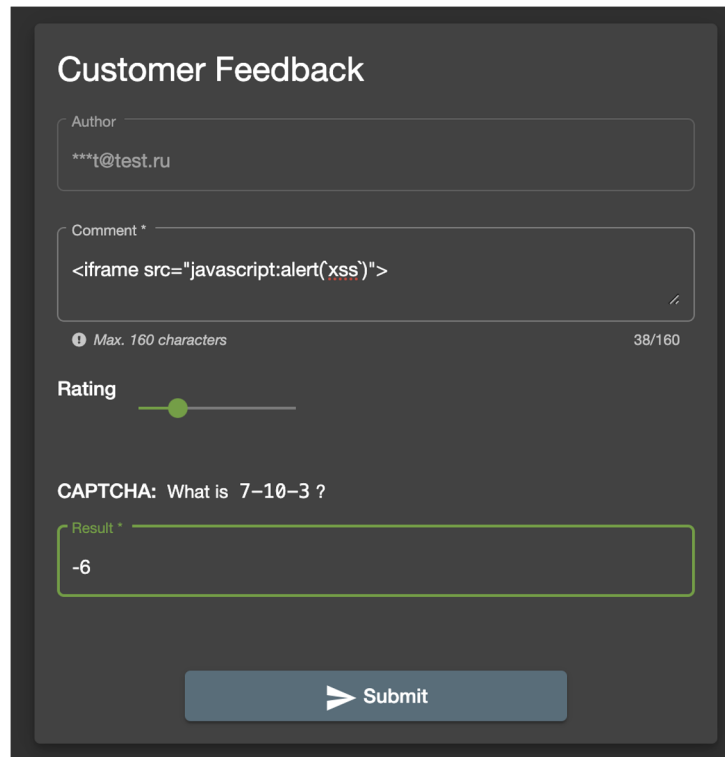
Хранимые XSS – XSS, в котором полезная нагрузка записывается в базу данных и выводится впоследствии без должной фильтрации.

Например, злоумышленник прячет вредоносный код в сообщении, которое оставляет внутри сайта (штатными средствами). Когда другой пользователь видит это сообщение у себя на экране, серверные скрипты исполняют код, содержащийся в нём.

- > Что нужно для выполнения?
- > Ошибочно написанные JS на сайте, которые выведут код из БД без модификации

Stored XSS пример

1. Нашли место, где потенциально можно сохранить какие-либо данные, которые сможет увидеть другой пользователь.
2. В идеале такое место, где мы сможем проверить, что именно он увидит, чтобы протестировать
3. Подбираем JS Payload



The image shows a web form titled "Customer Feedback" with a dark theme. It contains the following elements:

- Author:** A text input field containing the email address "****t@test.ru".
- Comment *:** A large text area containing the payload `<iframe src="javascript:alert('xss')">`. Below the text area, it indicates "Max. 160 characters" and "38/160".
- Rating:** A slider control with a green dot positioned at the left end.
- CAPTCHA:** A section titled "CAPTCHA: What is 7-10-3 ?" with a "Result *" input field containing the text "-6".
- Submit:** A button at the bottom right with a right-pointing arrow icon and the text "Submit".

Дополнительно

- Типы XSS: https://owasp.org/www-community/Types_of_Cross-Site_Scripting
- Решать задачи: https://pwning.owasp-juice.shop/companion-guide/latest/part2/xss.html#_perform_a_reflected_xss_attack

Same-origin policy

Действующие лица

<http://cs.itmo.xyz>



Наш вебсайт

<http://google.com>



Другой полезный
вебсайт

<http://evil.com>



Вебсайт
злоумышленника

Межсайтовое взаимодействие

<http://cs.itmo.xyz>



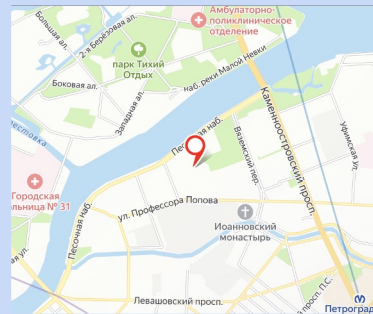
1. Запрашиваем скрипт

2. Получаем JS код

3. Исполняем, чтобы отрисовать карту

4. Запрашиваем изображение и данные

<https://api-maps.yandex.ru>



Межсайтовое взаимодействие

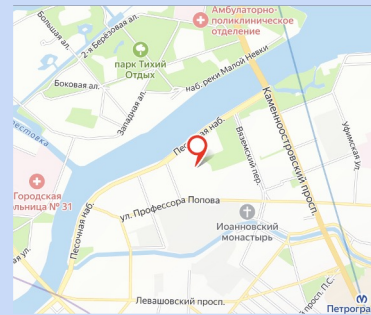
<http://evil.com>



1. Верни адрес проживания

2. Ок. Ты живешь тут: ...

<https://api.somemap.ru>



Авторизация по Cookie

Внимание

- Браузеры по-разному имплементируют системы защиты
- Каждые полгода что-нибудь меняется
- На самом деле всё уже не так, как я буду рассказывать :)
- Как оно на самом деле – будет в конце лекции
- Возможно, когда вы читаете эту презентацию в Google Chrome уже всё опять поменялось

Same-origin policy

Политика одинакового источника определяет как документ или скрипт, загруженный из одного источника, может взаимодействовать с ресурсом из другого источника. Это помогает изолировать потенциально вредоносные скрипты.

Origin – это совокупность **протокола, хоста и порта** (*scheme/host/port*)

Например,



http://cs.itmo.ru:80

Origin

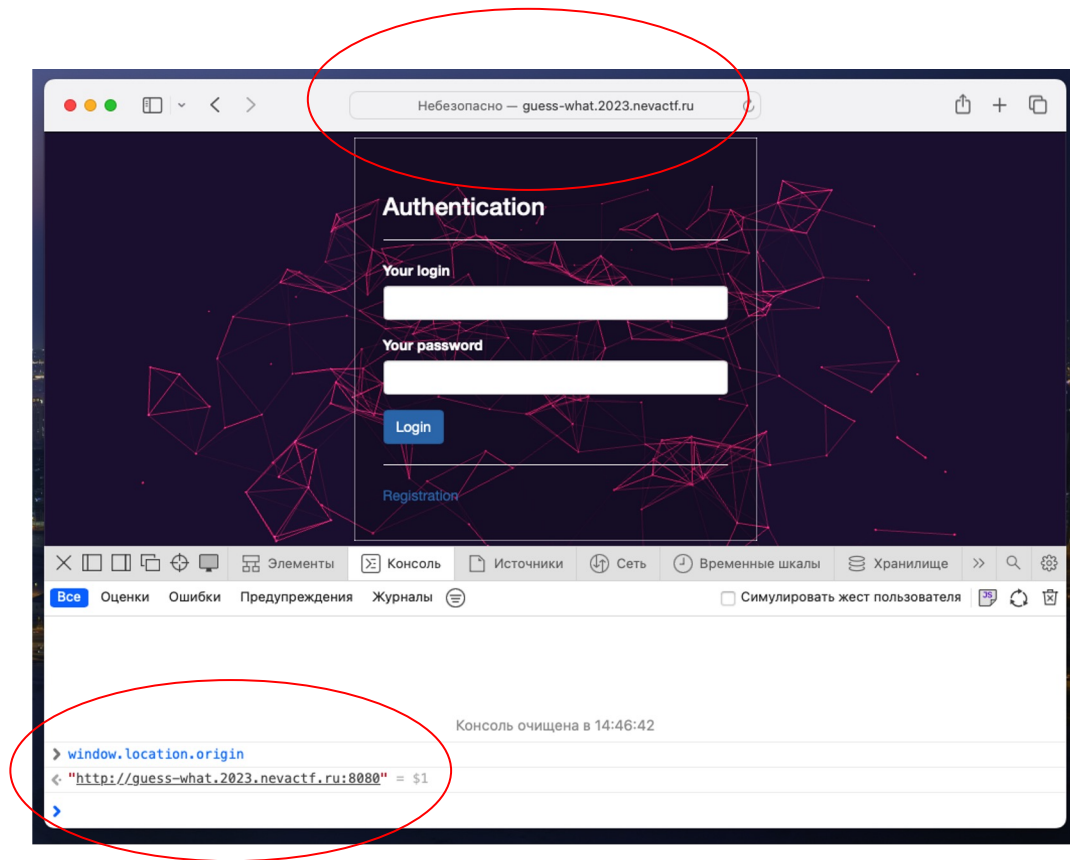
Если хотя бы 1 из трёх компонентов у двух сайтов не совпал – у них разный Origin. Путь к файлу при этом не имеет значения

Примеры origin-сравнений с URL *<http://store.company.com/dir/page.html>*:

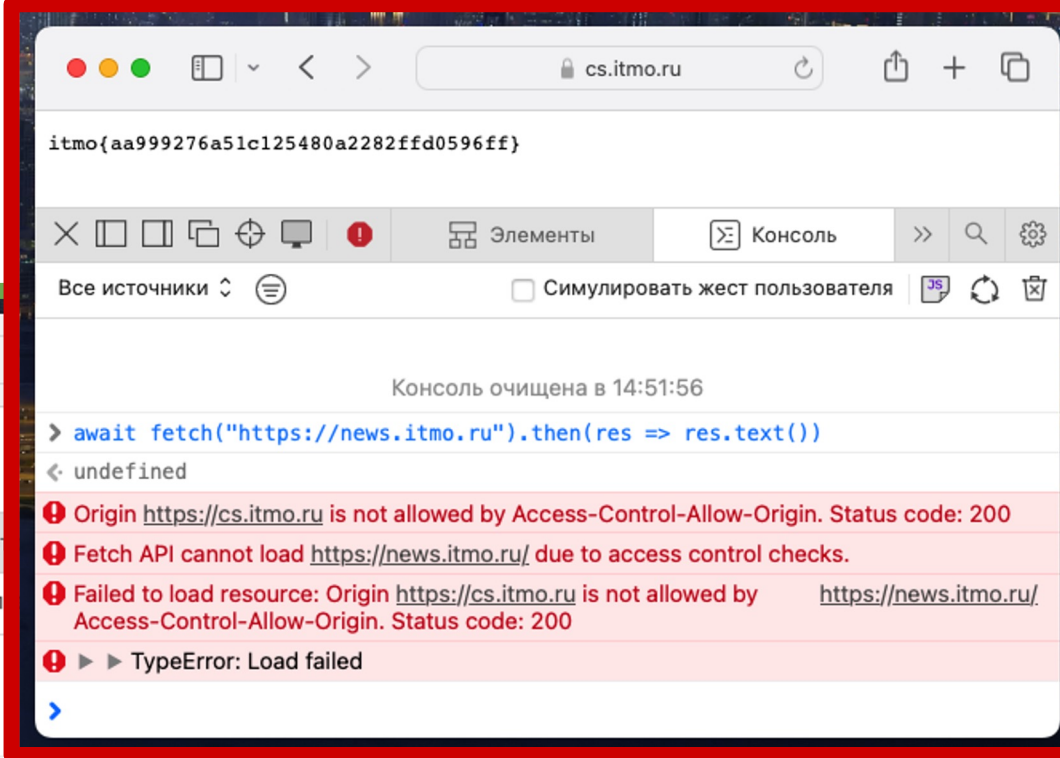
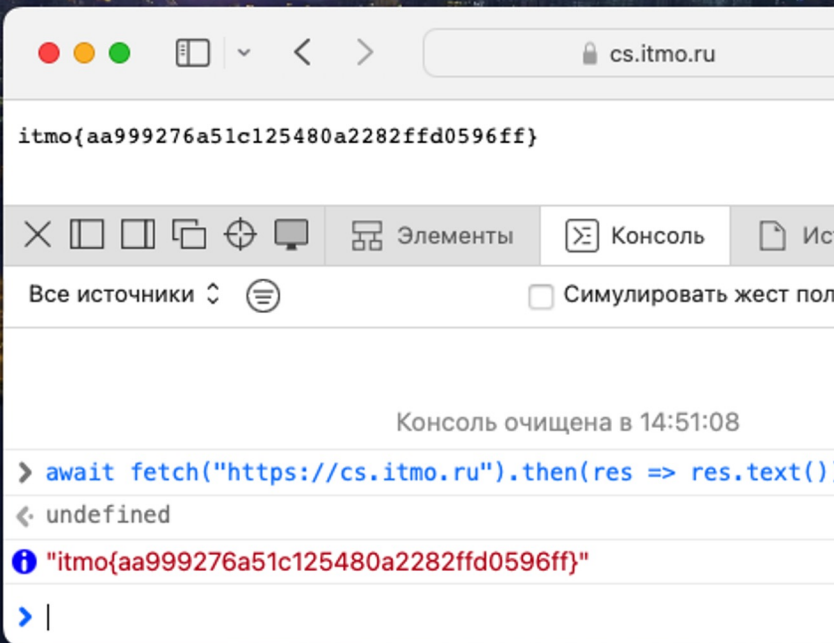
URL	Результат	Причина
<code>http://store.company.com/dir2/other.html</code>	Успех	
<code>http://store.company.com/dir/inner/another.html</code>	Успех	
<code>https://store.company.com/secure.html</code>	Ошибка	Разные протоколы: http и https
<code>http://store.company.com:81/dir/etc.html</code>	Ошибка	Разные порты: 80 и 81
<code>http://news.company.com/dir/other.html</code>	Ошибка	Разные хосты

Определяем Origin

```
> window.location.origin
```



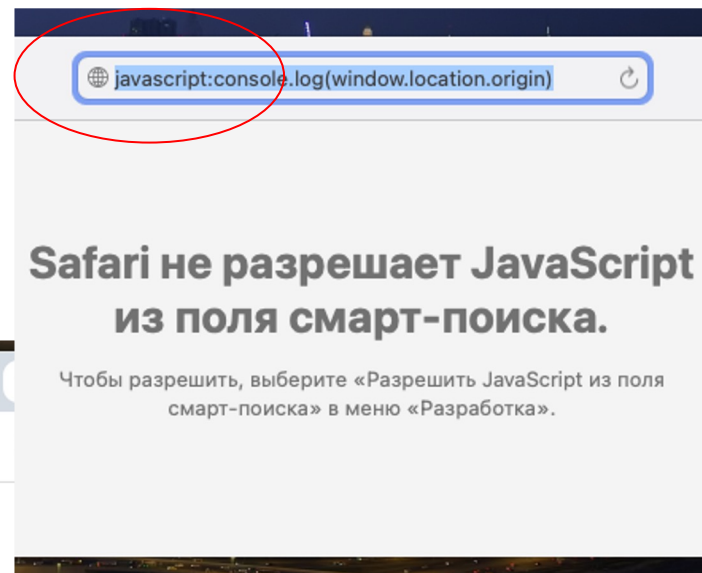
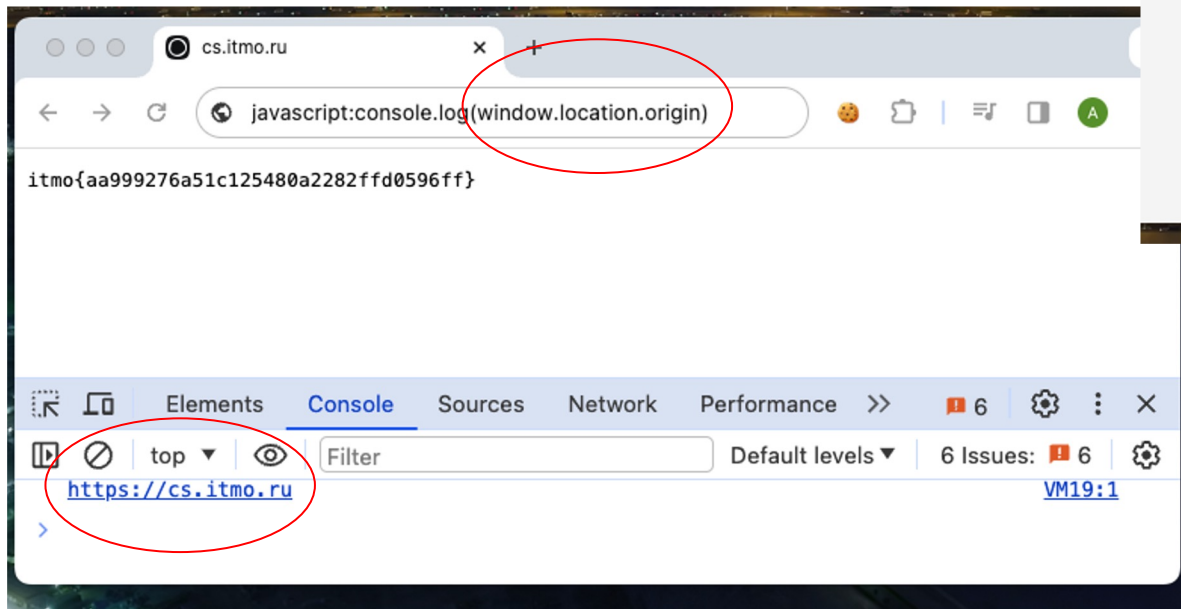
Проверяем защиту



```
await fetch("https://cs.itmo.ru").then(res => res.text())
```

Наследование Origins

URL: `javascript:console.log(window.location.origin)`



При copy-paste Chrome
вырежет часть строки:
javascript:

file:// Origin

```
<script>
```

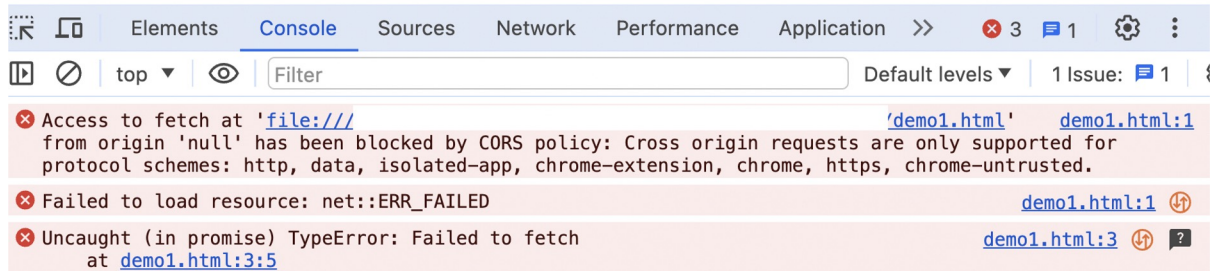
```
    document.write(`My origin is <code>${window.location.origin}</code>`)
```

```
</script>
```

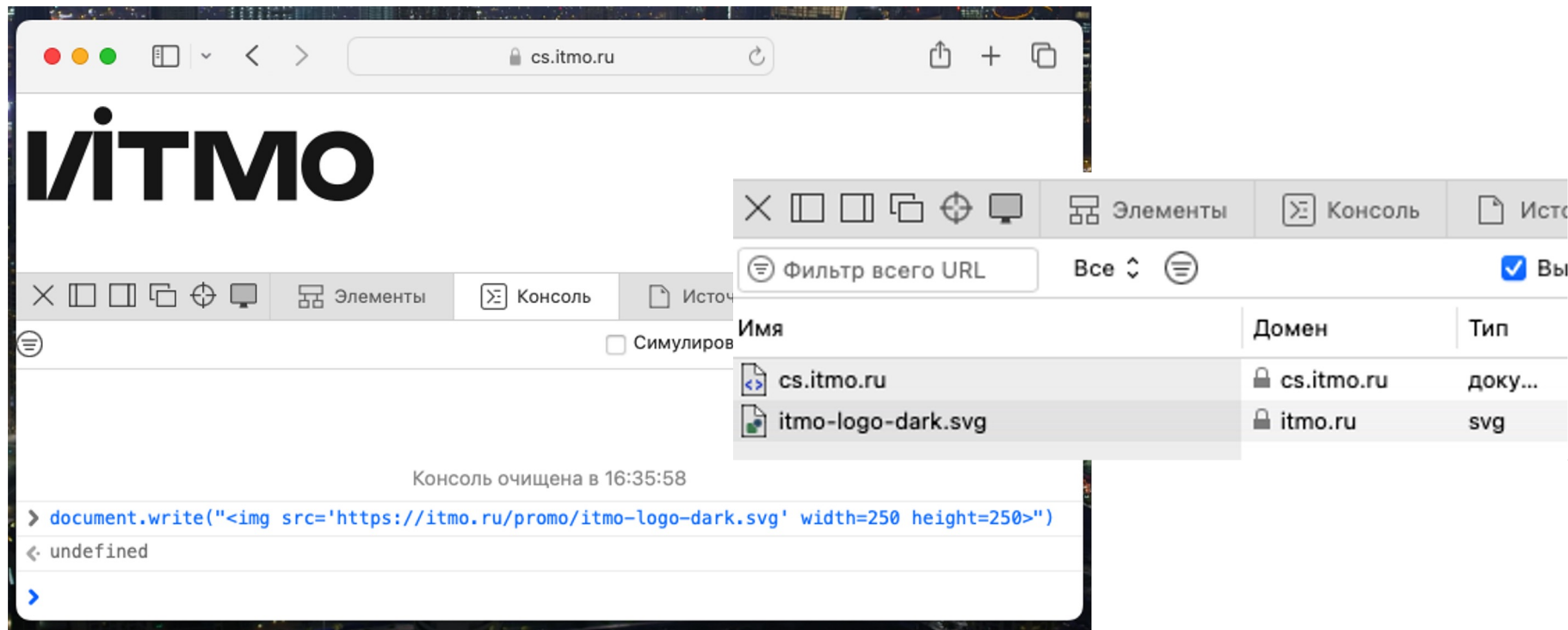
```
fetch("file:///.../demo1.html").then(res => res.text()).then(res =>
{document.write(res)});
```

Современные браузеры рассматривают Origin файлов, загруженных с помощью схемы *file:///*, как **opaque** (непрозрачное).

Если файл включает в себя другие файлы из той же папки, не предполагается, что они происходят из того же источника.

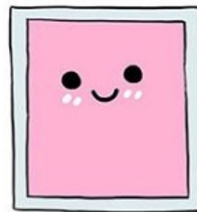


Постойте, а как же картинки подгружаются?

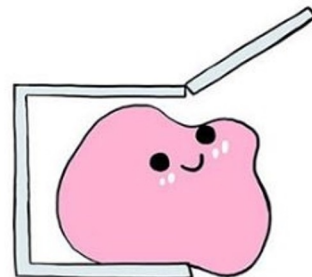


Cross-Origin Resource Sharing (CORS)

**ACCESS TO FETCH AT 'HTTP://LOCALHOST:3000/API' FROM
ORIGIN 'HTTP://127.0.0.1:3000' HAS BEEN BLOCKED BY CORS POLICY**



i think ill make a
web app

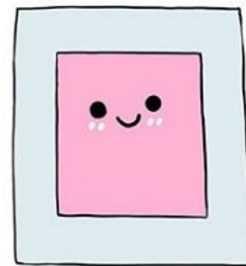


hey this isnt so bad



blocked by CORS policy:
the requested resource.

OWTORD.COM



NEVER AGAIN

Виды межсайтового взаимодействия (между разными Origin)

- Переход по ссылкам <a>
- Отправка форм <form>
- Вложение сущностей
 - JS <script src="..."></script>
 - CSS <link rel="stylesheet" href="...">
 - Изображения , Медиа <video> и <audio>
 - <object>, <embed>, <iframe>
 - Шрифты через @font-face URL
- XMLHttpRequest (XHR)
- Fetch API

*Как с помощью JavaScript
сделать куда-нибудь HTTP
запрос?*

***В целях безопасности браузеры ограничивают
cross-origin запросы, инициируемые скриптами.***

CORS

Cross-Origin Resource Sharing (CORS) — механизм, который вебсайт может использовать, чтобы явно разрешить доступ к себе от других **Origin**.

Такой запрос называется **Cross-origin HTTP request**.

- `fetch()` и `XMLHttpRequest`
- Web Fonts (`@font-face` в CSS)
- WebGL textures
- Images/video frames в canvas с использованием `drawImage()`
- CSS Shapes из изображений

“Простые запросы”

«Простой запрос» — это запрос, удовлетворяющий следующим условиям:

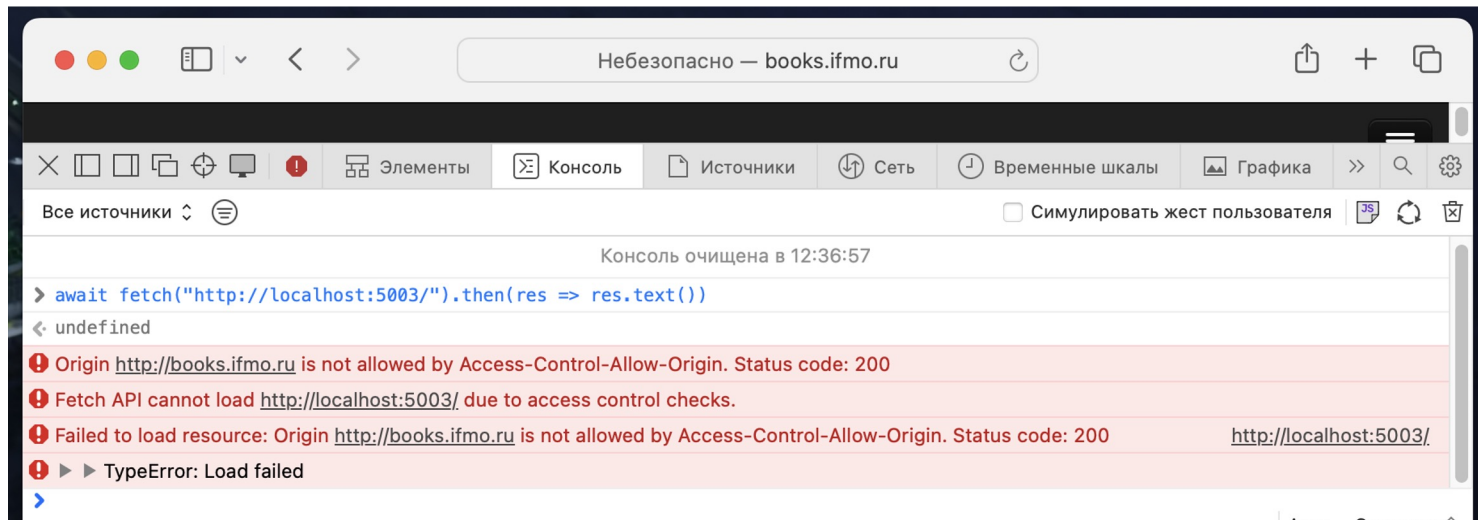
- Допустимые методы для запроса:
 - GET
 - HEAD
 - POST
- Допустимые значениями заголовка Content-Type:
 - application/x-www-form-urlencoded
 - multipart/form-data
 - text/plain
- Не использует нестандартных HTTP заголовков
 - <https://fetch.spec.whatwg.org/#forbidden-header-name>
 - <https://fetch.spec.whatwg.org/#cors-safelisted-request-header>

* плюс еще пара нюансов

https://developer.mozilla.org/ru/docs/Web/HTTP/CORS#%D0%BF%D1%80%D0%BE%D1%81%D1%82%D1%8B%D0%B5_%D0%B7%D0%B0%D0%BF%D1%80%D0%BE%D1%81%D1%8B

Пример “простого” запроса

Простые запросы физически выполняются, браузер просто не показывает результат



Destination	Protocol	Length	Key	Info
127.0.0.1	HTTP	497		GET / HTTP/1.1
127.0.0.1	HTTP/JSON	658		HTTP/1.1 200 OK , JSON (application/json)

Клиент передает Origin заголовок и ожидает ответа

Заголовок запроса **Origin** показывает откуда будет производиться загрузка. Он не включает в себя какую-либо информацию о пути, содержит в себе лишь **протокол, имя хоста и порт**.

```
✓ Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: localhost:5003\r\n
    Accept: */*\r\n
    Sec-Fetch-Site: cross-site\r\n
    Accept-Language: ru\r\n
    Accept-Encoding: gzip, deflate\r\n
    Sec-Fetch-Mode: cors\r\n
    Cache-Control: no-cache\r\n
    Origin: http://books.ifmo.ru\r\n
    User-Agent: Mozilla/5.0 (Macintosh;
    Referer: http://books.ifmo.ru/\r\n
    Connection: keep-alive\r\n
    Pragma: no-cache\r\n
    Sec-Fetch-Dest: empty\r\n
```

Client

Server



Ответ сервера: Access-Control-Allow-Origin

Заголовок ответа Access-Control-Allow-Origin показывает, может ли ответ сервера быть доступен коду, отправляющему запрос с данного источника origin.

Ответ, который указывает браузеру разрешить доступ к ресурсу из любого источника	Access-Control-Allow-Origin: *
Ответ, который указывает браузеру разрешить доступ к ресурсу только из источника <code>https://developer.mozilla.org</code>	Access-Control-Allow-Origin: <code>https://developer.mozilla.org</code>
Определяет в качестве источника "null" (но лучше вообще не прикреплять заголовок)	Access-Control-Allow-Origin: null

Заголовки ответа

- **Access-Control-Allow-Origin:** <origin> | * – разрешает указанному Origin доступ к нашему ресурсу
- **Access-Control-Allow-Credentials:** true – разрешает браузеру прикладывать Cookie при запросе к нашему ресурсу
- **Access-Control-Allow-Methods:** <method>[, <method>]* – разрешает отправлять запросы только с перечисленными HTTP методами
- **Access-Control-Allow-Headers:** <field-name>[, <field-name>]* – разрешает устанавливать кастомные заголовки только из перечисленных

Сложные запросы

В отличие от "простых запросов", "сложные запросы" сначала отправляют **Preflight** HTTP-запрос методом **OPTIONS** к ресурсу на другом домене, чтобы определить, является ли фактический запрос безопасным для отправки.

```
> await fetch("http://localhost:5003",{headers: {'X-CUSTOM': 'DEMO'}}).then(res => res.text())
```

⏪ undefined

❗ Origin <http://books.ifmo.ru> is not allowed by Access-Control-Allow-Origin. Status code: 200

❗ Fetch API cannot load <http://localhost:5003/> due to access control checks.

❗ Failed to load resource: Origin <http://books.ifmo.ru> is not allowed by Access-Control-Allow-Origin. Status code: 200

❗ ▶ ▶ TypeError: Load failed

Protocol	Length	Key	Info
HTTP	555		OPTIONS / HTTP/1.1
HTTP	261		HTTP/1.1 200 OK

Preflight request

```
555 OPTIONS / HTTP/1.1
442 HTTP/1.1 200 OK
513 GET / HTTP/1.1
682 HTTP/1.1 200 OK , JSON (application/json)
```

Теперь протестируем сервер, который устанавливает CORS заголовки

```
> await fetch("http://localhost:5003",{headers: {'X-CUSTOM': 'DEMO'}}).then(res => res.text())
```

OPTIONS / HTTP/1.1

Origin: http://books.ifmo.ru

Sec-Fetch-Site: cross-site

Sec-Fetch-Mode: cors

Access-Control-Request-Method: GET

Access-Control-Request-Headers: x-custom

Content-Length: 0

...

HTTP/1.1 200 OK

Allow: HEAD, OPTIONS, POST, GET

Access-Control-Allow-Origin: http://books.ifmo.ru

Access-Control-Allow-Headers: x-custom

Access-Control-Allow-Methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT

Vary: Origin

Content-Length: 0

...

GET / HTTP/1.1

X-CUSTOM: DEMO

Connection: keep-alive

...

HTTP/1.1 200 OK

Server: Werkzeug/3.0.1 Python/3.11.4

Content-Type: application/json

Content-Length: 626

Access-Control-Allow-Origin: http://books.ifmo.ru

Vary: Origin

Connection: close

Запросы с учётными данными

```
> await fetch("http://localhost:5003",{credentials: 'include'}).then(res => res.text())
```

```
⏪ undefined
```

❗ Credentials flag is true, but Access-Control-Allow-Credentials is not "true".

❗ Fetch API cannot load <http://localhost:5003/> due to access control checks.

❗ Failed to load resource: Credentials flag is true, but Access-Control-Allow-Credentials is not "true".

По идее, ставим `Access-Control-Allow-Credentials=true` и при запросе к такому ресурсу браузер отправит Cookie того ресурса, **к которому** идет запрос.

Но! --->

Тестируем на Chrome 123.0.6312.107

<https://developers.google.com/search/blog/2020/01/get-ready-for-new-samesitenone-secure?hl=ru>

В случае, когда cookie предназначен только для использования в собственном контексте, разработчик может запретить сторонний доступ к нему с помощью одной из этих настроек: **SameSite=Lax** или **SameSite=Strict**. Однако поскольку **далеко не все разработчики выполняют эту рекомендацию**, очень много файлов cookie даже в собственном контексте по-прежнему подвергается угрозам, таким как подделка межсайтовых запросов.

с февраля 2020 г. Нужно ставить *SameSite=None + Secure (работает только HTTPS)*

Тестируем на Chrome 123.0.6312.107

<https://cs.itmo.ru/webtech/>

Filter

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

⌵

Only show cookies with an issue

Name	Value	Domain	Pa...	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition ...	Priority
SECRET	ITMOTEST	cs.itmo.ru	/	Session	14		✓	None		Medium

Открываем консоль на любом сайте (например, <https://habr.com>)

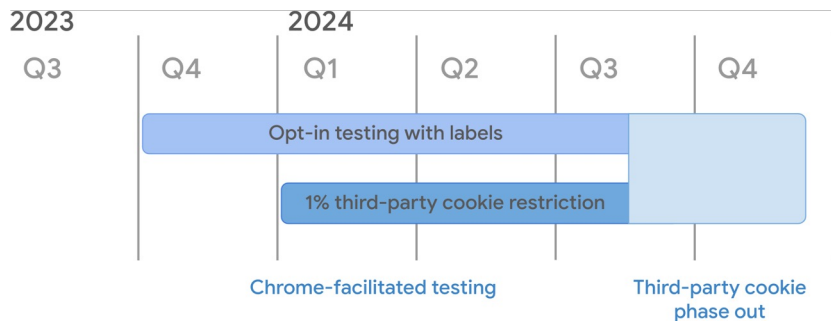
```
> await fetch("https://cs.itmo.ru/webtech/", {credentials:
'include'}).then(res => res.text())
```

Name	Headers	Preview	Response	Initiator	Timing	Cookies
webtech/	Accept-Encoding:					gzip, deflate, br, zstd
webtech/	Accept-Language:					ru,ru-RU;q=0.9,en-US;q=0.8,en;q=0.7
	Cache-Control:					no-cache
	Connection:					keep-alive
	Cookie:					SECRET=ITMOTEST
	Host:					cs.itmo.ru
	Origin:					https://habr.com
	Pragma:					no-cache
	Referer:					https://habr.com/ru/articles/top/daily/
	Sec-Ch-Ua:					"Google Chrome";v="123", "Not:A-Brand";v="8", "Chromium";v="123"

Про чипсы вместо печенек поговорим отдельно

⚠ Third-party cookie will be blocked. Learn more in the Issues tab.

<https://developers.google.com/privacy-sandbox/3pcd?hl=ru#report-issues>



Cookies Having Independent
Partitioned State
(CHIPS)

1

ЧИПСЫ

Новый атрибут cookie, Partitioned, позволяет разработчикам размещать файлы cookie в секционированном хранилище с отдельными банками файлов cookie для каждого сайта верхнего уровня.

Откройте для себя ЧИПС

2

API доступа к хранилищу

API доступа к хранилищу позволяет iframe запрашивать разрешения на доступ к хранилищу, если в противном случае доступ был бы запрещен настройками браузера.

Изучите API доступа к хранилищу

3

Сопутствующие наборы веб-сайтов

Наборы связанных веб-сайтов (RWS) — это способ для компании объявить взаимосвязи между сайтами, чтобы браузеры разрешили ограниченный доступ к сторонним файлам cookie для определенных целей.

Изучите похожие наборы веб-сайтов

4

API федеративного управления учетными данными

Веб-API для федерации удостоверений, сохраняющей конфиденциальность.

Откройте для себя направление Fe...

Same-Site

https://www.project.github.io:443
scheme eTLD+1 eTLD

Сайты, имеющие одинаковый протокол (scheme) и одинаковый eTLD+1, считаются "same-site".

URL (сравниваем с https://www.example.com:443)	Результат	Причина
https://www.evil.com:443	cross-site	Разные домены
https://login.example.com:443	same-site	Поддомены роли не играют
http://www.example.com:443	cross-site	Разные протоколы: http и https
https://www.example.com:80	same-site	Порты роли не играют
https://www.example.com	same-site	Совпадение, порт роли не играет

Тестируем на Chrome 123.0.6312.107

<https://cs.itmo.ru/webtech/>

Name	Value	Domain	Pa...	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition ...	Priority
SECRET	ITMOTEST	cs.itmo.ru	/	Session	14			Strict		Medium

Открываем консоль на **same-site** (например, <https://news.itmo.ru>)

```
> await fetch("https://cs.itmo.ru/webtech/", {credentials:
'include'}).then(res => res.text())
```

✕	Headers	Preview	Response	Ini
▼	{cookie: {SECRET: "ITMOTEST"}, getDa			
▼	cookie: {SECRET: "ITMOTEST"}			
	SECRET: "ITMOTEST"			
	getData: {}			
▶	headers: {Accept: "*/*", Accept-En			
	postData: null			

Content Security Policy (CSP)

Content Security Policy (CSP)

Content Security Policy (CSP) – это дополнительный уровень безопасности, позволяющий распознавать и устранять определённые типы атак, таких как XSS.

С помощью CSP можно указать, каким скриптам (изображениям/CSS/...) следует доверять, а каким нет.

Когда браузер пытается запустить скрипт из неизвестного источника, CSP блокирует его, если только он не находится в списке доверенных источников.

HTTP заголовок: `Content-Security-Policy: policy`

Формат заголовка

Разделяем директивы символом « ; »

Content-Security-Policy: <policy-directive>; <policy-directive>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy>

- **default-src** – используется для ресурсов, для которых не описано отдельных правил
- **object-src** – определяет источники для тегов <embed> и <object>
- **script-src** – источники для <script>, inline обработчиков (onclick)
- **img-src** – источники для изображений и favicon
- ...

Примеры

- Ограничить источники контента только исходным сервером

```
Content-Security-Policy: default-src 'self'
```

- Разрешить пользователям приложения вставлять в создаваемый ими контент картинки из любого источника, но при этом ограничить источники аудио- и видео-файлов списком доверенных провайдеров. Получение скриптов должно происходить только с конкретного сервера, содержащего доверенный код.

```
Content-Security-Policy: default-src 'self'; img-src *; media-  
src media1.com media2.com; script-src userscripts.example.com
```

CSP на GOOGLE.COM

Content-Security-Policy-Report-Only:

```
object-src 'none';base-uri 'self';script-src 'nonce-IhpLd92o7rdHDF3oN0Jv0Q' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri https://csp.withgoogle.com/csp/gws/other-hp
```

Будет
Deprecated



CSP на VK.COM

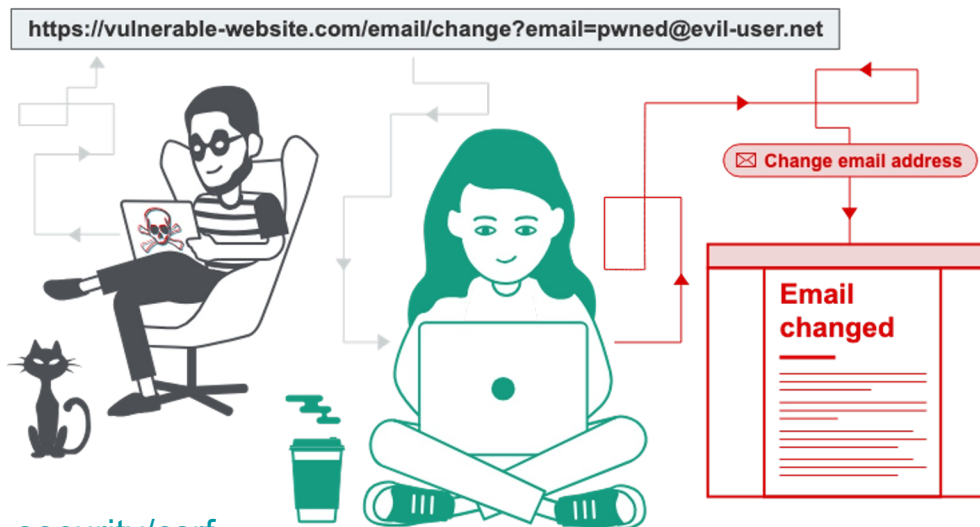
Content-Security-Policy:

```
default-src * data: blob: about: vkcalls:; script-src 'self'  
https://vk.com https://*.vk.com https://vk.ru https://*.vk.ru  
https://static.vk.me https://*.mail.ru  
https://platform.twitter.com  
https://cdn.syndication.twimg.com https://www.instagram.com  
https://connect.facebook.net https://telegram.org  
https://*.yandex.ru https://*.google-analytics.com  
https://*.youtube.com https://maps.googleapis.com  
https://translate.googleapis.com https://*.google.com  
<.....>  
'self' 'unsafe-inline'; report-uri /csp
```

Уязвимость CSRF

Cross-site request forgery (CSRF)

Подделка межсайтовых запросов (CSRF) – уязвимость, позволяющая злоумышленнику побудить пользователя выполнить действия, которые он не собирался выполнять. Она позволяет злоумышленнику частично обойти **same origin policy**.



Типовой сценарий уязвимости

```
Accept-Encoding: gzip, de
Accept-Language: ru,ru-RU
Cookie: SECRET=ITM0TEST;

message=%D0%A1%D0%BE%D0%B
```

1. Пользователь авторизован на сайте (например, **mail.ru**)
2. Пользователь перешел на вебсайт злоумышленников. Сайт содержит HTML форму вида:

```
<form action="http://mail.com/send" method="POST">
  <input type="hidden" name="message" value="Сообщение">
</form>
```
3. Автоматически при загрузке страницы JavaScript вызывает **form.submit**, отправляя таким образом форму на mail.com (это «простой запрос», поэтому Cookie будут приложены)
4. Из-за ошибки проектирования, сайт выполнит логику запроса **/send**

Защита

1. Обязательное использование **нестандартных заголовков**, чтобы запросы всегда были “сложными” и без учета CORS не выполнялись.
2. Генерировать случайные значения для каждой формы (**CSRF токен**). Такие токены нельзя угадать или подделать при отправке формы со стороннего сайта.
3. Если использовать Cookie с **SameSite=Strict**, то такая Cookie не будет отправлена со стороннего Origin

Итоги

1. Повторили основы синтаксиса **JavaScript**
2. Рассмотрели два вида уязвимостей **XSS**
3. Разобрались, что такое **Origin** и как определять одинаковый он или нет (**Same-origin policy**)
4. Изучили зачем нужен **Cross-Origin Resource Sharing (CORS)**, от каких атак он защищает и как усложняет работу Веб-программистов
5. Рассмотрели заголовок **Content Security Policy (CSP)** для защиты от XSS и небезопасных вложений
6. Изучили простейший сценарий уязвимости **CSRF**

Полезные ссылки

- Same origin – <https://web.dev/articles/same-site-same-origin?hl=ru>
- Same origin policy – https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
- CORS – <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- Same site cookies – <https://web.dev/articles/samesite-cookies-explained>
- Еще больше игр на XSS – <http://xss.school.sibears.ru/>
- Набор задач по Web безопасности – <https://pwning.owasp-juice.shop/>
- CSRF – <https://portswigger.net/web-security/csrf>

Вопросы?